

Drag & Drop en React

Guía completa de dnd kit — la librería estándar moderna para React

■ ■ ¿Qué es Drag & Drop y para qué sirve?

El **Drag and Drop** permite al usuario **arrastrar un elemento y soltarlo** en otro lugar para reordenar, mover o reorganizar contenido de forma visual e intuitiva.

■	Kanban / Trello: Arrastrar tarjetas entre columnas (To Do → In Progress → Done)
■	Listas ordenables: Reordenar tareas, pasos de un proceso, ítems de un menú
■	Subida de archivos: Arrastrar archivos desde el escritorio para subirlos
■	Tablas: Reorganizar columnas de una tabla arrastrando la cabecera
■	Page builders: Arrastrar bloques para construir formularios o páginas
■	Galerías de imágenes: Reordenar fotos arrastrándolas a la posición deseada

■ ¿Qué es dnd kit?

La librería de drag and drop **más moderna y recomendada para React**. Reemplaza a react-beautiful-dnd (sin mantenimiento activo). Modular, ligera, accesible y compatible con touch.

✓	Muy ligera y modular: Solo importas lo que necesitas
✓	Accesible por defecto: Soporta teclado y lectores de pantalla (ARIA)
✓	Touch y mouse: Funciona en móvil y escritorio sin configuración extra
✓	Sin dependencias externas: Solo React como peer dependency
✓	Muy flexible: Desde listas simples hasta Kanban multi-columna

■ ■ Instalación

```
npm install @dnd-kit/core @dnd-kit/sortable @dnd-kit/utilities
```

Paquete	Para qué sirve
@dnd-kit/core	Motor principal — DndContext, sensores, detección de colisiones
@dnd-kit/sortable	Utilidades para listas reordenables — useSortable, arrayMove
@dnd-kit/utilities	Helpers de CSS — CSS.Transform para animaciones suaves

■ Las piezas clave explicadas

Pieza	Qué hace
DndContext	El contenedor raíz que activa el D&D; en todo su interior. Recibe los eventos onDragStart, onDragEnd...

SortableContext	Indica qué elementos son reordenables y con qué estrategia (vertical, horizontal, grid)
useSortable()	Hook que da a cada item la capacidad de ser arrastrado: transform, listeners, setNodeRef...
arrayMove()	Utilidad que reordena el array al soltar: arrayMove(items, oldIndex, newIndex)
sensors	Detectan el gesto del usuario: PointerSensor (mouse/touch), KeyboardSensor (teclado)
collisionDetection	Algoritmo que decide cuándo un item 'entra' en una zona: closestCenter, closestCorners...
DragOverlay	Muestra una copia visual del elemento mientras se arrastra (efecto fantasma)

■ Ejemplo completo — lista reordenable

1. Componente item arrastrable — SortableItem

```
import { useSortable } from '@dnd-kit/sortable'
import { CSS } from '@dnd-kit/utilities'

function SortableItem({ id }: { id: string }) {
  const {
    attributes, // props de accesibilidad (role, aria-...)
    listeners, // eventos drag (onMouseDown, onTouchStart...)
    setNodeRef, // ref al elemento DOM
    transform, // posición durante el arrastre
    transition, // animación al soltar
    isDragging, // true mientras se arrastra
  } = useSortable({ id })

  const style = {
    transform: CSS.Transform.toString(transform),
    transition,
    opacity: isDragging ? 0.4 : 1, // semitransparente al arrastrar
    cursor: isDragging ? 'grabbing' : 'grab',
  }

  return (
    <div ref={setNodeRef} style={style} {...attributes} {...listeners}>
      {id}
    </div>
  )
}
```

2. Componente padre — SortableList

```
import { DndContext, closestCenter, PointerSensor,
KeyboardSensor, useSensor, useSensors } from '@dnd-kit/core'
import { SortableContext, sortableKeyboardCoordinates,
verticalListSortingStrategy, arrayMove } from '@dnd-kit/sortable'
import { useState } from 'react'

export function SortableList() {
  const [items, setItems] = useState(['Tarea 1', 'Tarea 2', 'Tarea 3'])

  const sensors = useSensors(
    useSensor(PointerSensor),
    useSensor(KeyboardSensor, {
      coordinateGetter: sortableKeyboardCoordinates,
    })
  )

  function handleDragEnd(event) {
    const { active, over } = event // active = lo arrastrado, over = destino

    if (active.id !== over?.id) {
      setItems(prev => {
        const oldIndex = prev.indexOf(active.id)
        const newIndex = prev.indexOf(over.id)
        return arrayMove(prev, oldIndex, newIndex) // reordena el array
      })
    }
  }

  return (
    <DndContext
      sensors={sensors}
      collisionDetection={closestCenter}
      onDragEnd={handleDragEnd}
    >
      <SortableContext items={items} strategy={verticalListSortingStrategy}>
        {items.map(id => <SortableItem key={id} id={id} />)}
      </SortableContext>
    </DndContext>
  )
}
```

■ Casos avanzados

Múltiples listas (Kanban)	Usa varios SortableContext (uno por columna) + onDragOver para detectar cuando un item cruza de una lista a otra. El estado es un objeto { columna: items[] }.
Drag Handle	En lugar de poner {...listeners} en el contenedor entero, ponlos solo en un icono de arrastre (■). El resto de la tarjeta puede tener clicks normales sin activar el drag.
DragOverlay	Envuelve el item visual dentro de . Mientras se arrastra, el original queda en su sitio (semitransparente) y el overlay sigue al cursor con animación suave.
Restricciones de movimiento	Usa restrictToVerticalAxis o restrictToParentElement de @dnd-kit/modifiers para limitar el arrastre solo a un eje o dentro de un contenedor.

■ Documentación oficial con ejemplos interactivos: **dndkit.com** — tiene demos en vivo para listas, Kanban, grids y casos avanzados.